INTRODUCTION

1. INTRODUCTION

SCRAPHERE: The Vehicle Scrap Management System is a web-based application developed to simplify and modernize the process of vehicle scrapping. Traditionally, vehicle owners face difficulties in finding reliable scrap centers, comparing offers, and managing the paperwork involved in disposing of old or damaged vehicles. This project aims to address these challenges by providing a centralized digital platform that connects vehicle owners with authorized scrap centers.

The system allows users to register, create profiles, and list vehicles that are no longer in use or have reached the end of their life cycle. Registered scrap centers can view vehicle listings, submit offers, and manage scrap requests through the platform. Users can review the offers received and choose the most suitable scrap center for their vehicle.

An administrator oversees the entire system by managing categories, brands, vehicle models, locations, users, and scrap center registrations. The platform also includes features such as complaint management, feedback submission, profile management, and request tracking, ensuring transparency and effective communication among all stakeholders.

Developed using the Django framework, the Vehicle Scrap Management System provides a secure, user-friendly, and efficient solution for vehicle disposal. By digitizing the scrapping process, the system helps reduce manual effort, improves accessibility, and promotes environmentally responsible vehicle recycling practices.

Objective

The main objective of the Vehicle Scrap Management System is to provide a digital platform that simplifies and streamlines the process of vehicle scrapping for both vehicle owners and scrap centers.

Specific Objectives

- To provide an online platform for vehicle scrapping
- Enable vehicle owners to easily register and submit vehicles for scrapping without visiting multiple scrap centers.
- To connect vehicle owners with authorized scrap centers
- Create a centralized system where scrap centers can view vehicle details and offer quotations for scrap vehicles.
- To improve transparency in the scrapping process
- Allow users to compare offers from different scrap centers and choose the most suitable option.
- To reduce manual paperwork
- Digitize vehicle scrap requests, approvals, and communication, making the process faster and more efficient.
- To facilitate effective management
- Provide administrative tools for managing users, scrap centers, vehicle categories, brands, models, complaints, and feedback.
- To enhance user experience
- Offer a user-friendly interface for submitting requests, tracking status, managing profiles, and communicating with scrap centers.
- To promote environmentally responsible vehicle disposal
- Encourage proper recycling and disposal of end-of-life vehicles through registered and authorized scrap centers.
- To ensure secure and organized data management
- Store vehicle, user, and scrap center information securely while maintaining data accuracy and accessibility.

ABOUT THE ORGANIZATION

2. ABOUT THE ORGANIZATION

Progressive Software Solutions & Training is a fast developing, well established Information Technology enterprise with highly talented and efficient staff meeting the needs of clients throughout the world. The company is so popular that the clients feed the company with repeat orders, having faith in the capabilities. The staff includes management and engineering personnel, programmers, web developers, graphic designer, business analysts, technical writers etc. apart from the regular commercial and administrative hands. The motto of this reliable software provider is 'DO BEST; GET BEST' Progressive Software Solutions & Training committed to provide what they have promised. Progressive Software Solutions & Training strives for the satisfaction of their clients and dedicates themselves for the better services to their customers.

Progressive Software Solutions & Training are famous in providing high quality, cost cutting, requirement oriented and value added web and software solutions with remarkable cyber services. Progressive Software Solutions & Training are specialist in developing ever rewarding internet existence for ambitious companies which are in the hangover of swift moving internet facilities. Their aim is to grow with such organizations in a very fruitful manner. Their main concentration is to collect business oriented technological know-how and utilize them for the wellbeing of business concerns.

Progressive Software Solutions & Training experts in various technologies are committed to deliver effective and creative service to customers.

Services

- Software Development
- Web Development
- SEO Services (Search Engine Optimization)

SYSTEM ANALYSIS

3. SYSTEM ANALYSIS

3.1 EXISTING SYSTEM

The existing system of police is a traditional, manual, and reactive approach to law enforcement. Here police officers manually write the complaints and the communication between officers, stakeholders etc are limited. The reactive is large and it has limited channels for public to report incidents. In this time eventually there are some kinds of websites and applications are there.

Disadvantage

- Time consuming

3.2 PROPOSED SYSTEM

Here the proposed system is make it computerized and help public to interact with police officers. It is a comprehensive, integrated solution designed to enhance public safety, prevent and investigate crimes, and provide emergency response services. This help the public and they can share their problem or even an incident to the authorities. Basically the project describes how to manage good performance and better services for user. Here we provide more security and privacy compare to existing system.

3.3 SOFTWARE REQUIREMENTS SPECIFICATION

HARDWARE SPECIFICATION

The use of sufficient hardware is very important for the proper working of any software. Following are the hardware required by the system.

Processor	Intel Core i5
Processor Speed	1.5 GHz or higher
System Bus	32 Bit
Memory	1 GB RAM or higher
Hard Disk	40 GB or higher
Monitor	15.6” LCD Monitor
Keyboard	104 Keys

SOFTWARE SPECIFICATION

Software is a part of a computer system that consists of encoded information or computer instructions, in contrast to the physical hardware from which the system is built. Following are the software required by the system.

Operating System	Windows 11
IDE	Visual Studio code
Front End	PYTHON DJANGO, HTML 5
Back End	SQL3
Web Server	APACHE
Browser	Internet Explorer, Mozilla Firefox, Chrome

3.4 FEASIBILITY STUDY

The main objective of this study is to determine whether the proposed system is feasible or mainly there are three types of feasibility study to which the proposed system is subjected to as described below:

- Economic feasibility
- Technical feasibility
- Behavioral feasibility

The proposed system must be evaluated from a technical viewpoint first, and if technically feasible then their impact on organization must be accessed. If compatible, the behavioural system can be devised. Then those must be tested for economic feasibility.

ECONOMIC FEASIBILITY

Economic analysis is used for evaluating the cost effectiveness of a proposed system. Economically saying, our application is never cost related. It is easy to implement, and the user can conveniently use our application without any kind of exceptions or errors. Our application is very simple to interact is economically feasible.

TECHNICAL FEASIBILITY

The technical requirements of our system are highly affordable. There is no any difficulty in migrating from the existing to our proposed system. It is platform independent the application will run smoothly on any kernel without causing any trouble to end user.

BEHAVIORAL FEASIBILITY

The system is intended for personal uses. Since the software is simple and provides a good Graphical User Interface, it provides high level user.

3.5 DATA FLOW DIAGRAM

To start the system design, something analogue to the architecture blue print as a starting point to design is required. It is a way to focus on functions rather than physical implementation. One such tool is a DFD.

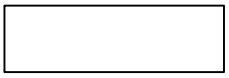
Structured analysis is a set of techniques and graphical tools that help the analyst to develop a new kind of system specification that are easily understandable to the user. DFD's show the major decompositions of the system functions and their interfaces. The DFD is graphic and presents a picture of what is being specified and is conceptually easy to understand presentation of the application.

One important feature of DFD's is that it is logical rather than physical. The elements of the system do not depend on vendor or hardware. They specify in precise, concise manner the working of the system and how it hangs together.

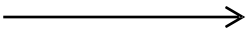
DFD is the graphic representation of data movement process, and files used in support of an information system. There are several rules of thumb used in drawing DFDs.

- Process should be named and numbered for easy references.
- The direction of flow is from top to bottom and from left to right.
- When a process is imported in the lower levels details, they must be numbered.
- Process and data flow names have the first letter of each word must be a capital letter.

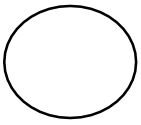
The symbol used in DFD are shown be



Source or destination of data



Data flow

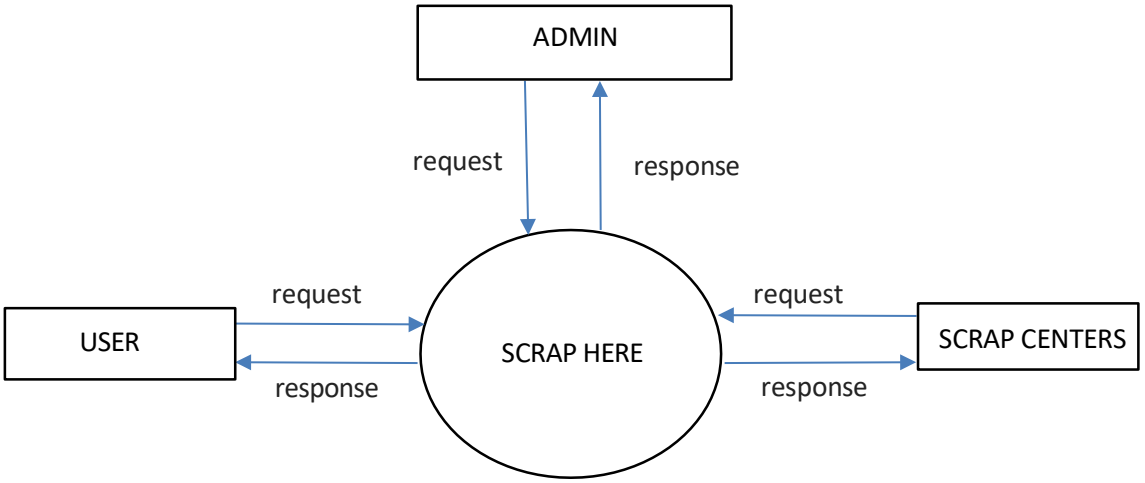


Process that transform data flow



Data store

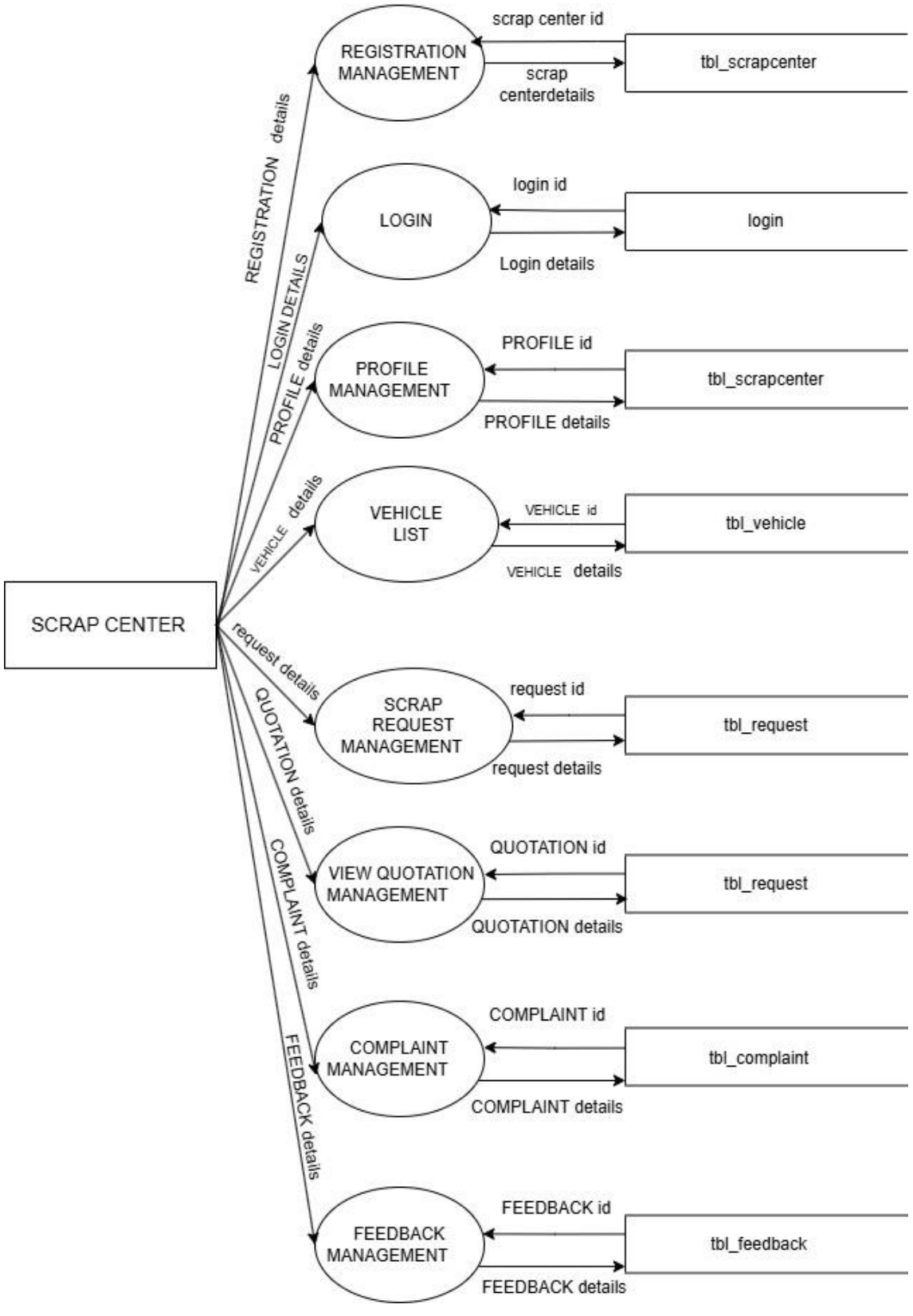
CONTEXT LEVEL



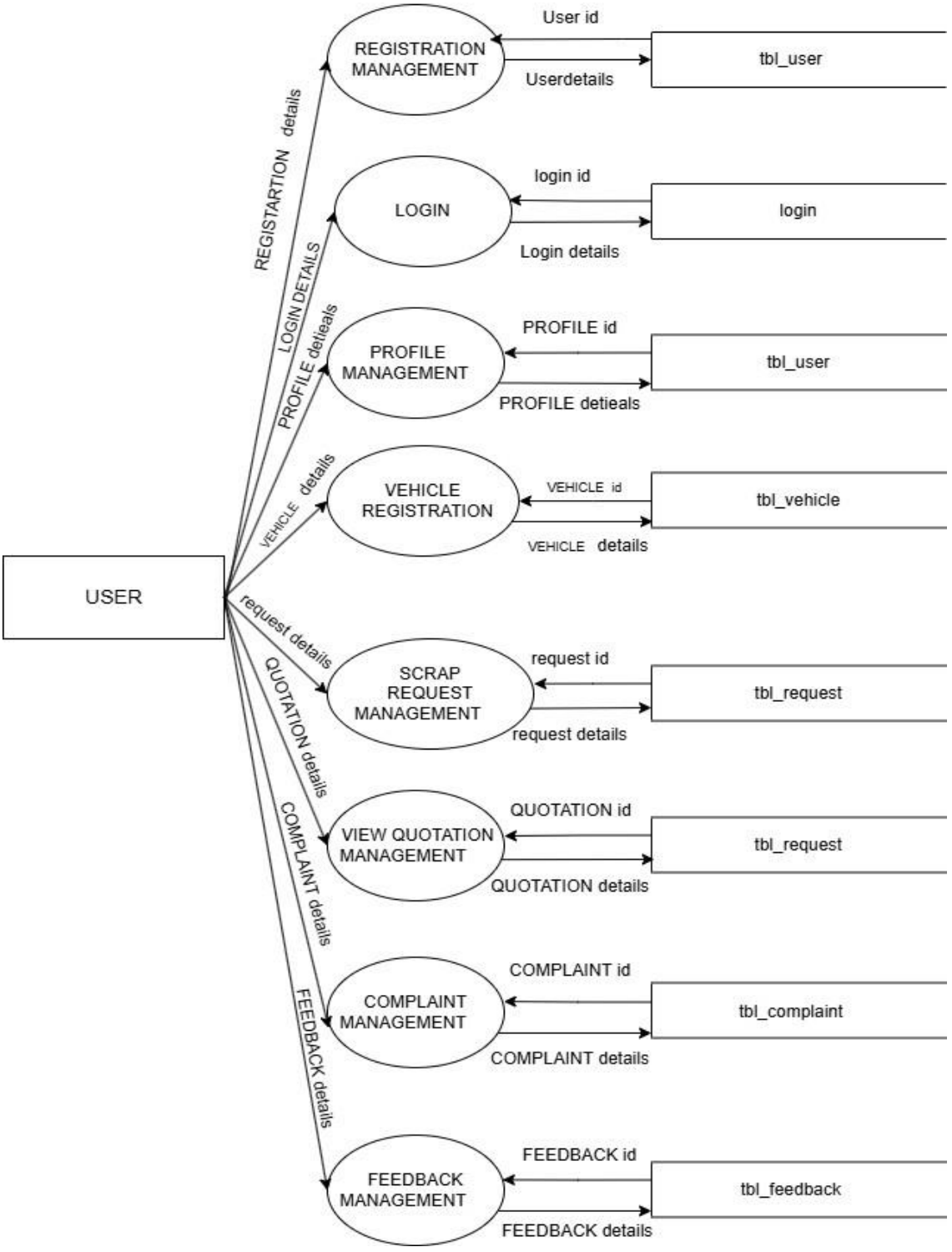
LEVEL 1 FOR ADMIN



LEVEL 1 FOR SCRAP CENTER



LEVEL 1 FOR USER



SYSTEM DESIGN

4. SYSTEM DESIGN

System design is the creative act of invention developing new inputs, a database offline files, method, procedures and output for processing business data to meet organization objectives. The design phase focuses on the detailed implementation of the system recommended in the feasibility study. The design phase is a transition from user-oriented document to a document oriented to the programmers or database personnel. Characteristics of a well-designed system are:

- Accessibility
- Decision making ability
- Economy
- Flexibility
- Reliability
- Simplicity

The design will determine the success of the system. System design is based on the information gathered during system analysis. System design goes through two phases of development.

- Logical design- DFD shows the logical flow of a system and defines the boundaries of the system. For the candidate the system it describes the input, outputs, databases and procedures-all in a format that meets the user's requirements.
- Physical design - This produces the working system by defining the design specification that tells programmers exactly what the candidate system must do.

4.1 INPUT DESIGN

Input design is the link that ties the information system into the world of its users. The input design involves determining what the inputs are, how the data should be performed, how to validate data, how minimize data entry and how to provide a multi-user facility inaccurate.

Input data are the most common cause of error in data processing. Errors entered by data entry operators can be controlled by input design. Input design is the process of converting user.

Originated inputs to a computer-based format. Input data are collected and organized into groups of similar data. Once identified, appropriate input media are selected for processing. All the input data validated in the order and if any data violates any condition, the user is warned by a message. If the data satisfies all the conditions, then it is transferred to the appropriate tables in the database.

4.2 OUTPUT DESIGN

Output design involves specifying how production of on-screen reports and page based reports will occur. Output may occur to database or file for storing information entered or also for use by other systems. Computer output is the most important and direct source of information to the user. Output design is a very important phase because the output needs to be in an attractive manner. Efficient, intelligible output design improves the system relationship with the user and help in decision making. A major form of the output is the hard copy from the printer and screen reports. Printouts are designed around the output requirements of the user. Allowing the user to view to the sample screen is important because the user is ultimate judge of the quality of output. The output model of this system is the user-friendly window. These user-friendly windows are meant for the purpose of easy view of the stored information.

4.3 TABLE DESIGN

A database is a collection of data. Database design refers to the design of the tables used to store data. The database involves name of records, data item with its name, type and size. In the design of the database program first we must thoroughly look into the requirements of the program for the design of database. Then we must design how much tables are required in the database. Thereafter as per the requirement of the end users we can decide which fields that must be in this table. As per a general rule a provision must be taken in the design for the future enhancement of the program. Some of important tables are:

TABLE STRUCTURE

1. **Table name: tbl_admin**
   Table description: To store the details of admin

Field Name	**Data Type**	**Constraints (Primary key, foreign key, not null, null)**	**Field Description**
admin_id	int(10)	Primary Key	Admin id
admin_name	varchar(30)	Not null	Name of admin
admin_email	varchar(50)	Not null	Email of admin
admin_password	varchar(30)	Not null	Password of admin

2. **Table name: tbl_district**
   Table description: To store the details of district

Field Name	**Data Type**	**Constraints (Primary key, foreign key, not null, null)**	**Field Description**
district_id	int(10)	Primary key	District id
district_name	varchar(30)	Not null	Name of the district

3. **Table name: tbl_place**

Table description: To store the details of place

Field Name	Data Type	Constraints (Primary key, foreign key, not null, null)	Field Description
place_id	int(10)	Primary key	Place id
place_name	varchar(30)	Not null	Name of the place
district_id	int(10)	Foreign key (tbl_district)	District id

4. **Table name: tbl_category**

Table description: To store the details of category of the complaint

Field Name	Data Type	Constraints (Primary key, foreign key, not null, null)	Field Description
category_id	int(10)	Primary key	Category id
category_name	varchar(30)	Not null	Name of Category

5. Table name: tbl_brand

Table description: To manage brand details

Field Name	Data Type	Constraints (Primary key, foreign key, not null, null)	Field Description
brand_id	int(10)	Primary key	Brand id
brand_name	varchar(30)	Not null	Name of brand

6. Table name: tbl_vehiclemodel

Table description: To store vehicle models

Field Name	Data Type	Constraints (Primary key, foreign key, not null, null)	Field Description
model_id	int(10)	Primary key	Model id
model_name	varchar(30)	Not null	Name of model
brand_id	int(10)	Foreign key (tbl_brand)	Brand id

7. Table name: tbl_user

Table description: To store the details of the user

Field Name	Data Type	Constraints (Primary key, foreign key, not null, null)	Field Description
user_id	int(11)	Primary key	User id
user_name	varchar(25)	Not null	Name of user
user_email	varchar(25)	Not null	Email of user
user_password	varchar(25)	Not null	Password of user
user _gender	varchar(20)	Not null	Gender of user
user_phone	varchar(300)	Not null	Contact of user
user_address	varchar(300)	Not null	Address of the user
user _status	varchar(30)	Not null	Status of user
place_id	int(10)	Foreign key (tbl_place)	Place id

8. Table name: tbl_scrapcenter

Table description: To store the details of scrap centers

Field Name	Data Type	Constraints (Primary key, foreign key, not null, null)	Field Description
center_id	int(10)	Primary key	Center id
center_name	int(10)	Foreign key (tbl_user)	Name of scrap center
center_email	date	Not null	Email of center
center_phone	time	Not null	Contact number
center_address	int(10)	Foreign key (tbl_policestation)	Address of center
center_status	varchar(20)	Not null	Status of center
place_id	int(10)	Foreign key (tbl_place)	Place id
center_password	int(30)	Not null	Password of center

9. Table name: tbl_vehicle

Table description: To store vehicle details

Field Name	Data Type	Constraints (Primary key, foreign key, not null, null)	Field Description
vehicle_id	int(10)	Primary key	Children Id
user_id	int(10)	Foreign key (tbl_user)	User id
category_id	int(10)	Foreign key (tbl_category)	Category id
brand_id	int(10)	Foreign key (tbl_brand)	Brand id
Model_id	int(10)	Foreign key (tbl_vehiclemodel)	Model id
vehicle_year	int(4)	Not null	Manufacturing year

10. Table name: tbl_request

Table description: To store scrap requests

Field Name	Data Type	Constraints (Primary key, foreign key, not null, null)	Field Description
request_id	int(10)	Primary key	Participant Id
user_id	int(10)	Foreign key (tbl_user)	Children Id
vehicle_id	date	Not null	Date of safetybutton
center_id	time	Not null	Time of safetybutton
request_date	int(10)	Foreign key (tbl_policestation)	Policestation Id
request_status	varchar(20)	Not null	Status of safetybutton

11. Table name: tbl_complaint

Table description: User complaints

Field Name	Data Type	Constraints (Primary key, foreign key, not null, null)	Field Description
complaint_id	int(10)	Primary key	Complaint Id
complaint_file	varchar(250)	Not null	file of the complaint
complaint_content	varchar(250)	Not null	Content of the complaint
complaint_date	date	Not null	Date of the complaint
user_id	int(10)	Foreign key (tbl_user)	User Id
children_id	int(10)	Foreign key (tbl_children)	Children Id
policestation_id	int(10)	Foreign key (tbl_policestation)	Policestation Id
complaint_status	varchar(25)	Not null	Status of the complaint

12. Table name: tbl_feedback

Table description: Feedback for our website

Field Name	Data Type	Constraints (Primary key, foreign key, not null, null)	Field Description
feedback_id	int(10)	Primary key	Feedback Id
feedback_content	varchar(300)	Not null	Content of feedback
feedback_date	date	Not null	Date of the feedback
user_id	int(10)	Foreign key (tbl_user)	User Id
children_id	int(10)	Foreign key (tbl_children)	Children Id
policestation_id	int(10)	Foreign key (tbl_policestation)	Policestation Id

4.4 PROCESS DEIGN

There are 4 modules in the system.

ADMIN

1. ADMIN

- Login
- District Management
- Place Management
- Category Management
- Subcategory Management
- Brand Management
- Model Management
- User Verification
- Scrap Center Verification
- Complaint & Feedback Management

2. User

- Registration
- Login
- Profile management
- Vehicle registration
- Scrap request management
- View quotation
- Complaint management
- Feedback management

3. Scrap Center

- Registration
- Login
- Profile Management
- Vehicle List
- Scrap Request Management
- View Quotation
- Complaint Management
- Feedback Management

SYSTEM TESTING AND IMPLEMENTATION

3. SYSTEM TESTING & IMPLEMENTATION

Testing is the process of examining the software to compare the actual behavior of the system with the expected behavior. The major objective of software testing is to ensure that faults and errors are identified and eliminated before the system is deployed. To achieve this goal, the tester executes the program with the intention of finding defects and verifying that the software functions correctly. Although testing cannot guarantee the complete absence of errors, it helps to ensure that the system operates reliably and efficiently.

System testing is defined as the process of detecting defects in software and verifying that the developed system satisfies the specified requirements. It is one of the most important phases in software development because it provides the final opportunity to identify and rectify errors that may have occurred during the design and coding stages. Testing is carried out to ensure the correctness, reliability, maintainability, and performance of the Vehicle Scrapping Management System.

The Vehicle Scrapping Management System was developed to provide an efficient platform for managing vehicle scrapping activities. The system allows users to register vehicles, submit scrap requests, receive quotations from scrap centers, and communicate through complaints and feedback. Administrators can manage locations, categories, brands, models, users, and scrap centers effectively. Therefore, thorough testing was required to ensure that all modules functioned accurately and efficiently.

System testing is the first stage of implementation and is aimed at ensuring that the developed system works correctly before it is made available for actual use. Testing plays a vital role in the success of the system. It is based on the assumption that if all individual modules perform correctly and are properly integrated, then the overall objectives of the system will be achieved successfully. A series of tests were conducted on the proposed system before it was made ready for user acceptance testing.
The testing steps carried out for the Vehicle Scrapping Management System are:

- Unit Testing
- Integration Testing
- Validation Testing
- Output Testing
- Acceptance Testing

System testing provides assurance that the software, once validated, can be integrated with all other system elements successfully. It verifies whether all modules have been combined properly and whether the overall system functionality and performance requirements have been achieved. After integrating the User Module, Scrap Center Module, and Admin Module, validation testing was performed on the complete system. The results showed that all modules worked together effectively and satisfied the overall objectives of the Vehicle Scrapping Management System..

- # UNIT TESTING

Unit testing is carried out module-wise, with each module being treated as an independent unit. Attention is focused on individual modules separately in order to identify coding errors, logical errors, and database-related issues. This approach helps ensure that each module performs its intended function correctly before integration with other modules.

Various test cases were prepared and executed for each module of the Vehicle Scrapping Management System. The results obtained were compared with the expected outputs to verify that the modules functioned according to the specified requirements.

The following modules were subjected to unit testing:

- User Registration Module
- User Login Module
- Vehicle Registration Module
- Scrap Request Management Module
- Quotation Management Module
- Complaint Management Module
- Feedback Management Module
- Scrap Center Registration Module
- Scrap Center Login Module
- Admin Management Module

During unit testing:

• Module interfaces were tested to ensure that information flowed correctly between forms, modules, and the database.
• Boundary conditions were tested to ensure that the modules operated properly within the specified limits and restrictions.
• All independent execution paths through the control structures were executed to ensure that every statement in the module was executed at least once.
• Input validation was tested to ensure that invalid or incomplete data was rejected appropriately.
• Database operations such as insertion, updation, deletion, and retrieval were verified.
• Error handling paths were tested to ensure that the system responded correctly to unexpected or invalid inputs.

The results of unit testing showed that all modules performed according to the specified requirements and produced the expected outputs.

- # INTEGRATION TESTING

Integration testing is a systematic technique for constructing the program structure while simultaneously conducting tests to uncover errors associated with interfacing between modules. The individually unit-tested modules were integrated to form a complete system structure as specified in the design phase. Incremental integration testing was adopted in this project.

The purpose of integration testing is to verify the communication and interaction between different modules of the Vehicle Scrapping Management System. After ensuring that each module functioned correctly in isolation, the modules were integrated and tested together to identify any interface errors, data transfer issues, or functional inconsistencies.

The following integrations were tested:

- User Registration and Login Module
- User Module and Vehicle Registration Module
- Vehicle Registration and Scrap Request Module
- Scrap Request and Quotation Management Module
- User Module and Complaint Management Module
- User Module and Feedback Management Module
- Scrap Center Module and Quotation Management Module
- Admin Module and User Verification Module
- Admin Module and Scrap Center Verification Module

During integration testing:

• Data flow between modules was verified to ensure that information was transferred accurately.
• Database interactions between integrated modules were tested to ensure consistency and integrity of stored data.
• Interfaces between User, Scrap Center, and Admin modules were tested thoroughly.
• Communication between modules was checked to ensure that all integrated functions operated according to system requirements.
• Error conditions occurring during module interaction were identified and corrected.

The integration testing process confirmed that all modules worked together successfully. The Vehicle Scrapping Management System performed as an integrated application without any significant interface errors, and all system functionalities operated according to the specified requirements.

- # VALIDATION TESTING

Validation testing is performed to ensure that the developed software satisfies all user requirements and functions according to the specified objectives. It is carried out after integration testing to verify that the complete system behaves as expected in a real-world environment.

The users and scrap centers conduct testing on the system and provide feedback regarding its functionality and usability. The developer is not directly involved during this testing phase, making it possible to evaluate the software under actual operating conditions. Any issues identified during testing are reported, analyzed, and corrected through suitable modifications to the system.

In the Vehicle Scrapping Management System, client-side and server-side validation techniques are implemented to ensure that only valid data is accepted by the system.

Validation testing was performed on various input fields such as:

- User Name
- Email Address
- Mobile Number
- Password
- Vehicle Registration Number
- Vehicle Details
- Scrap Request Details
- Quotation Amount

Incorrect or invalid values entered in these fields are detected and rejected by the system. Appropriate error messages are displayed to guide the user in providing valid information. For example, invalid email formats, incorrect mobile numbers, empty mandatory fields, and invalid passwords are not accepted by the system.

Validation testing also ensured that:

• All mandatory fields were properly validated.
• Invalid data entries were rejected.
• Appropriate error messages were displayed for incorrect inputs.
• User registration and login processes accepted only valid credentials.
• Vehicle registration and scrap request submissions were validated before storage in the database.

The results of validation testing confirmed that the Vehicle Scrapping Management System correctly validates user inputs and prevents invalid data from entering the database, thereby improving system reliability and data integrity.

.

- **OUTPUT TESTING**

Output testing is performed to ensure that the Vehicle Scrapping Management System generates accurate and user-friendly outputs. The outputs displayed on the screen were verified against user requirements and expected results.The details related to vehicle registration, scrap requests, quotations, complaints, feedback, and verification processes were tested and found to be correct.

The output format was clear, well-organized, and easy for users to understand.Since all outputs were generated accurately and satisfied user requirements, no major modifications were required after output testing.

- **ACCEPTANCE TESTING**

Acceptance testing is performed on the completed system to ensure that it satisfies user requirements and operates correctly in the intended environment. Each test case is executed to verify the functionality and usability of the system, resulting in either a pass or fail outcome.

Acceptance testing is carried out by end users to determine whether the Vehicle Scrapping Management System meets their expectations. The system was developed according to user requirements, and the interfaces were designed to be simple, clear, and user-friendly.

The users successfully performed activities such as vehicle registration, scrap request submission, quotation viewing, complaint management, and feedback submission without any difficulties. The system passed the acceptance testing phase successfully and was accepted by the users as it fulfilled all specified requirements.

.

- # DEBUGGING

Debugging is the process of identifying and removing errors or bugs from software to ensure its proper functioning. It helps in detecting coding errors, logical mistakes, and runtime issues that may affect system performance.

During the development of the Vehicle Scrapping Management System, debugging was carried out to identify and correct errors in user registration, login, vehicle registration, scrap request management, quotation management, and database operations. Various debugging techniques and tools were used to locate the source of errors and resolve them effectively.

The debugging process involved:

- Reproducing the problem.
- Identifying and analyzing the cause of the error.
- Examining the program state and variable values.
- Correcting the identified error.
- Retesting the system to ensure that no new errors were introduced.
- Through proper debugging, the system was made more reliable, stable, and efficient, ensuring
- smooth operation of all modules.

IMPLEMENTATION

Implementation is the final and most important phase of system development. It involves converting the designed system into a working application through coding, testing, user training, and deployment. During this phase, the Vehicle Scrapping Management System takes its actual form and becomes ready for practical use.

The implementation phase is concerned with translating the system design into source code using Python Django, HTML, CSS, JavaScript, and SQLite database. Before deployment, extensive testing was carried out to ensure that the system operated according to the specified requirements without any major errors.

The developed system was tested by users, and necessary modifications were made based on their feedback. After successful testing and validation, the system was implemented and made operational. The implementation process ensured smooth transition from the existing manual process to the computerized Vehicle Scrapping Management System.

The implementation stage involved the following activities:

- Testing the developed system.
- Detection and correction of errors.
- User training and guidance.
- Database setup and configuration.
- System deployment and operation.
- Evaluation of system performance.

The Vehicle Scrapping Management System was successfully implemented and found to be efficient, reliable, and user-friendly. The system fulfilled all specified requirements and performed all operations accurately.

IMPLIMENTATION PLANS:

➢ Testing the system with sample user, vehicle, and scrap request data.
➢ Detection and correction of errors identified during testing.
➢ Making necessary modifications and improvements to the system.
➢ Comparing the developed system with the existing manual process to verify efficiency and accuracy.
➢ Installation and configuration of required software utilities, including Python Django, SQLite Database, and supporting tools.
➢ Training and guidance for administrators, users, and scrap center personnel regarding system operation.
➢ Database setup and deployment of the Vehicle Scrapping Management System.
➢ Monitoring and evaluation of system performance after implementation.

SECURITY TECHNOLOGIES AND POLICIES

.

4. SECURITY TECHNOLOGIES AND POLICIES

SYSTEM SECURITY

Threats to System Security

Any act, event, or circumstance that poses a risk to the system and its data is known as a threat. Management must be aware of various security threats in order to protect system resources through proper policies, security measures, and technology.

A threat may be intentional, such as unauthorized access by hackers, or accidental, such as hardware failures and natural disasters. The Vehicle Scrapping Management System stores important information related to users, vehicles, scrap requests, quotations, complaints, and feedback. Therefore, adequate security measures are necessary to protect the system from potential threats.

The major threats to system security include:

- ➢ Unauthorized access to the system and modification or deletion of stored data.
- ➢ Attempts to gain access to user accounts through password attacks.
- ➢ Uploading malicious software such as viruses, worms, and Trojan programs.
- ➢ Hardware failures that may result in data loss.
- ➢ Power fluctuations and excess electrical supply affecting system operation.
- ➢ Natural disasters such as floods, earthquakes, and fire accidents.
- ➢ Database corruption or accidental deletion of records.
- ➢ Network-related attacks that may affect system availability and performance.

Proper security mechanisms, user authentication, data backup procedures, and access control techniques are implemented to minimize the impact of these threats and ensure the safe operation of the Vehicle Scrapping Management System.

SECURITY TECHNOLOGIES AND POLICIES

Security and maintenance are important aspects of any software system. Maintenance involves modifying and updating the software to ensure that it continues to function efficiently and meets changing user requirements. It includes correcting errors, improving performance, and making necessary enhancements to the system after deployment.

The Vehicle Scrapping Management System requires regular maintenance to ensure smooth operation and reliability. Maintenance activities include fixing reported issues, updating system features, improving user interfaces, and adapting the software to changes in hardware and software environments.
Since the system stores important information such as user details, vehicle information, scrap requests, quotations, complaints, and feedback, adequate security measures are essential. Security mechanisms are implemented to protect the system from

unauthorized access, data loss, and malicious activities.

The following security measures are incorporated in the system:

- ➢ User authentication through valid username and password.
- ➢ Access control based on user roles such as Administrator, User, and Scrap Center.
- ➢ Protection of sensitive data stored in the database.
- ➢ Validation of user inputs to prevent invalid or malicious data entry.
- ➢ Secure management of vehicle, request, and quotation records.
- ➢ Backup and recovery mechanisms to prevent data loss.
- ➢ Monitoring and maintenance of system performance and security.

The system allows access only to authorized users with valid login credentials. These security technologies and policies help ensure the confidentiality, integrity, and availability of information within the Vehicle Scrapping Management System.

SYSTEM SECURITY MEASURES

Security is an important factor in the Vehicle Scrapping Management System. Since the system stores user details, vehicle information, scrap requests, quotations, complaints, and feedback, appropriate security measures are implemented to prevent unauthorized access and protect confidential data.

1. Passwords

The most common and effective method of security is password protection. A password is a secret combination of characters used for user authentication and access control. It verifies the identity of users before granting access to the system.
In the Vehicle Scrapping Management System, every Administrator, User, and Scrap Center must log in using a valid username and password. This prevents unauthorized users from accessing sensitive information and performing restricted operations.

2. OTP Verification

One-Time Password (OTP) verification is implemented as an additional layer of security. During registration, password reset, or account verification processes, an OTP is sent to the registered email address of the user.
The user must enter the correct OTP within a specified time period to complete the verification process. Since the OTP is generated dynamically and can only be used once, it significantly reduces the risk of unauthorized account access.

Benefits of OTP Verification:

- ➢ Provides an additional layer of authentication.
- ➢ Prevents unauthorized account access.
- ➢ Enhances account security during registration and password recovery.
- ➢ Ensures that only genuine users can access system services.

3. Data Security

All important information related to users, vehicles, scrap requests, quotations, complaints, and feedback is stored securely in the database. Access to data is restricted based on user roles and permissions.

4. Input Validation

The system validates all user inputs before storing them in the database. Invalid, incomplete, or malicious data entries are rejected, ensuring data integrity and system reliability.
These security measures help maintain the confidentiality, integrity, and availability of information within the Vehicle Scrapping Management System.
.

MAINTENANCE

5. MAINTENANCE

Maintenance is the process of modifying a software system after its deployment in order to correct faults, improve performance, adapt to changing requirements, and enhance system functionality. Maintenance is an important phase of the software development life cycle because it ensures that the system continues to operate efficiently and effectively over time.

In the Vehicle Scrapping Management System, maintenance includes correcting coding and design errors, updating system documentation, improving user interfaces, and enhancing existing features. It also involves updating the system according to user feedback and changing technological requirements.
Maintenance activities help to keep the system reliable, secure, and user-friendly. Any issues reported by users can be identified and corrected promptly. The system can also be upgraded by adding new features and improving existing functionalities whenever required.

The future updates and maintenance can be carried out even after the system has been delivered to users. Maintenance is not limited to bug fixing alone; it also includes adapting the software to changing environments, improving performance, and meeting new user requirements.

Software maintenance is a broad activity that includes error correction, capability enhancement, optimization, and system improvement. Since changes are inevitable, appropriate mechanisms must be adopted for evaluating, controlling, and implementing modifications. The primary objective of maintenance is to preserve and enhance the value of the software over time.

The categories of maintenance are:

Corrective Maintenance:

Corrective maintenance involves identifying and fixing errors, bugs, and defects discovered after the system is deployed. It ensures the smooth functioning of the Vehicle Scrapping Management System.

Adaptive Maintenance:

Adaptive maintenance is performed to modify the system according to changes in the operating environment, technology, hardware, software, or user requirements.

Perfective Maintenance:

Perfective maintenance focuses on improving system performance, usability, efficiency, and adding new features based on user feedback and future requirements.

CORRECTIVE MAINTENANCE

Corrective maintenance refers to the modifications made to the Vehicle Scrapping Management System to correct defects and errors discovered after deployment. These defects may arise from design errors, logical errors, coding mistakes, database inconsistencies, or improper handling of user inputs.

Design errors may occur when system requirements are misunderstood or implemented incorrectly. Logic errors may result from incorrect conditions, calculations, or program flow. Coding errors are caused by mistakes during the implementation of program logic and database operations. In addition, data processing errors and performance issues may also require corrective maintenance.

When a system failure or unexpected behavior is identified, corrective actions are taken to restore normal system operation. The errors are analyzed, corrected, and thoroughly tested before being applied to the live system. This helps maintain the reliability and efficiency of the Vehicle Scrapping Management System.

Proper corrective maintenance ensures that issues related to user registration, vehicle management, scrap requests, quotation processing, OTP verification, complaints, and feedback handling are resolved quickly and effectively. It also minimizes the impact of errors on users and improves overall system performance.
.

ADAPTIVE MAINTENANCE

Adaptive maintenance involves modifying the Vehicle Scrapping Management System to accommodate changes in its operating environment. The environment includes hardware platforms, software technologies, databases, operating systems, security requirements, and organizational policies that may change over time.

As technology evolves, the system may require updates to remain compatible with newer versions of software and hardware. Adaptive maintenance ensures that the application continues to function efficiently under these changing conditions.

This type of maintenance may include upgrading the Django framework, updating database configurations, migrating the system to a new server, enhancing OTP and authentication mechanisms, or adapting the application to new operating systems and web browsers. Changes in government regulations related to vehicle scrapping, environmental policies, or data security requirements may also require modifications to the system.

Adaptive maintenance helps the Vehicle Scrapping Management System remain reliable, compatible, and effective in a continuously changing technological and operational environment.

PERFECTIVE MAINTENANCE

Perfective maintenance refers to modifications made to improve the performance, efficiency, usability, and functionality of the Vehicle Scrapping Management System. As users continue to use the system, new requirements and suggestions may arise that help enhance the overall quality of the application.

This type of maintenance focuses on improving existing features and adding new functionalities to meet changing user needs. Enhancements may include improving the user interface, optimizing database operations, increasing system performance, generating additional reports, and introducing advanced features such as real-time notifications, online payment integration, or mobile application support.
Perfective maintenance also aims to make the software more efficient, user-friendly, well-documented, and easier to maintain. By continuously improving the system, the Vehicle Scrapping Management System can provide better services to users, scrap centers, and administrators while adapting to future requirements and technological advancements.

CONCLUSION

6. CONCLUSION

a. Scope for Future Enhancement

The Vehicle Scrapping Management System has been successfully developed and implemented according to the requirements specified by the users. The system has been thoroughly tested and found to be functioning correctly without major errors. It provides an efficient platform for managing vehicle scrapping activities, including vehicle registration, scrap request management, quotation handling, complaint management, feedback management, and administrative controls.

The system reduces manual work, improves data accuracy, and provides better communication between users, scrap centers, and administrators. Through proper authentication, OTP verification, and secure database management, the system ensures reliable and secure operation.

Although the current system satisfies all major requirements, future enhancements can further improve its functionality and efficiency. Some possible future enhancements include:

- ➢ Online payment integration for scrap transactions.
- ➢ Mobile application development for Android and iOS platforms.
- ➢ Real-time notifications through SMS and email services.
- ➢ GPS-based scrap center location and navigation services.
- ➢ AI-based vehicle valuation and scrap price prediction.
- ➢ Integration with government vehicle registration databases.
- ➢ Advanced reporting and analytics for administrators.
- ➢ Cloud-based deployment for improved scalability and accessibility.
- ➢ Real-time tracking of scrap request status.
- ➢ Enhanced security features using advanced authentication mechanisms.

The future updates and maintenance can be performed even after the system has been delivered to users. By modifying existing components and adding new functionalities, the system can continue to evolve according to changing user requirements and technological advancements. Maintenance not only includes fixing errors but also adapting the software to new environments, improving performance, and enhancing overall user experience.

Thus, the Vehicle Scrapping Management System provides a strong foundation for efficient vehicle scrapping management and has significant potential for future growth and enhancement.

BIBLIOGRAPHY

7. BIBLIOGRAPHY

[1] Pressman, Roger S. **Software Engineering: A Practitioner's Approach**, McGraw-Hill Education, 8th Edition, 2014.

[2] Silberschatz, Abraham, Korth, Henry F., and Sudarshan, S. **Database System Concepts**, McGraw-Hill Education, 6th Edition, 2011.

[3] Elmasri, Ramez and Navathe, Shamkant B. **Fundamentals of Database Systems**, Pearson Education, 7th Edition, 2016.

[4] Holovaty, Adrian and Kaplan-Moss, Jacob. **The Definitive Guide to Django**, Apress Publications.

[5] Sommerville, Ian. **Software Engineering**, Pearson Education, 10th Edition, 2015.

REFERENCES

1. https://www.djangoproject.com
2. https://docs.djangoproject.com
3. https://www.python.org
4. https://www.sqlite.org
5. https://www.w3schools.com
6. https://developer.mozilla.org
7. https://getbootstrap.com
8. https://stackoverflow.com
9. https://www.geeksforgeeks.org
10. https://github.com

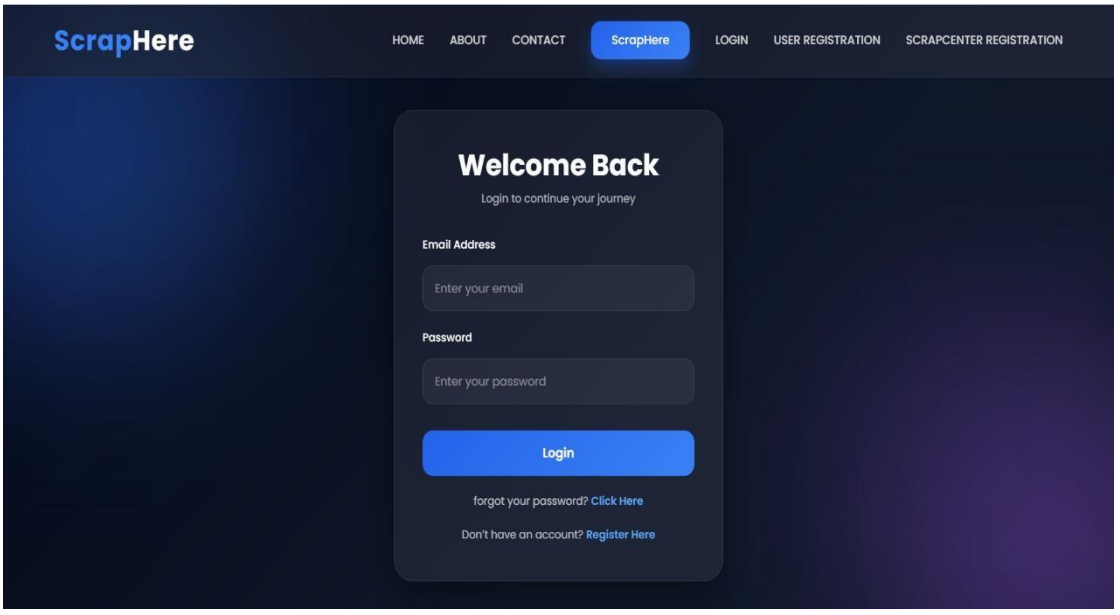
APPENDIX

8. APPENDIX

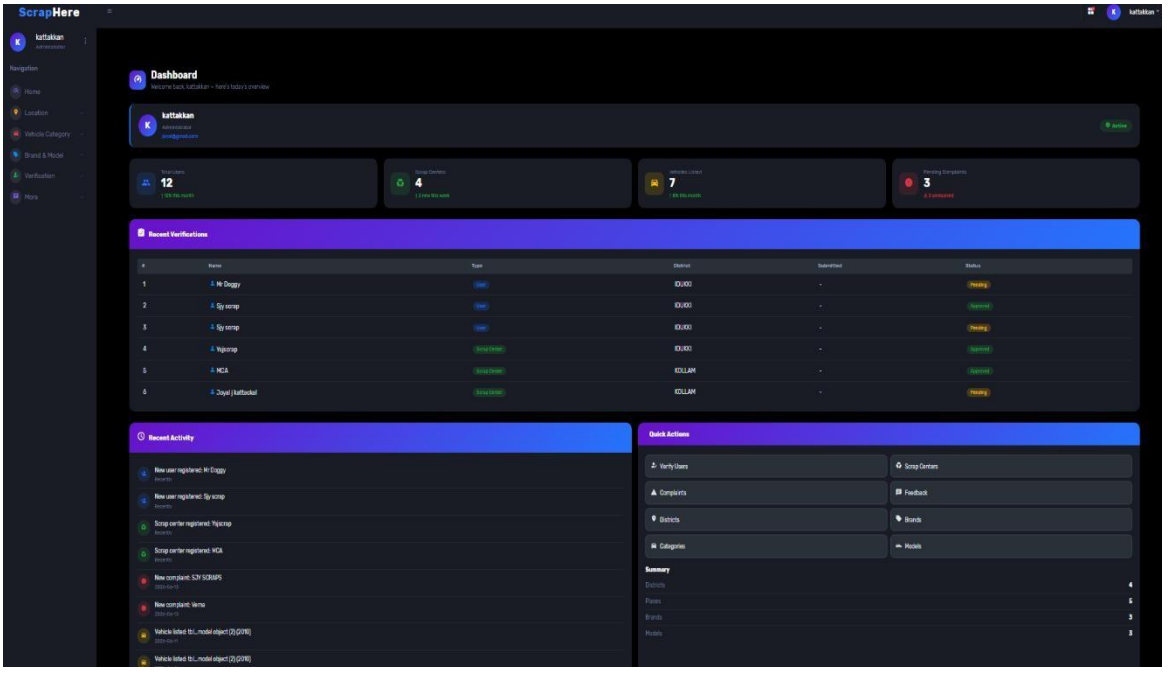
a. Screenshort

ADMIN

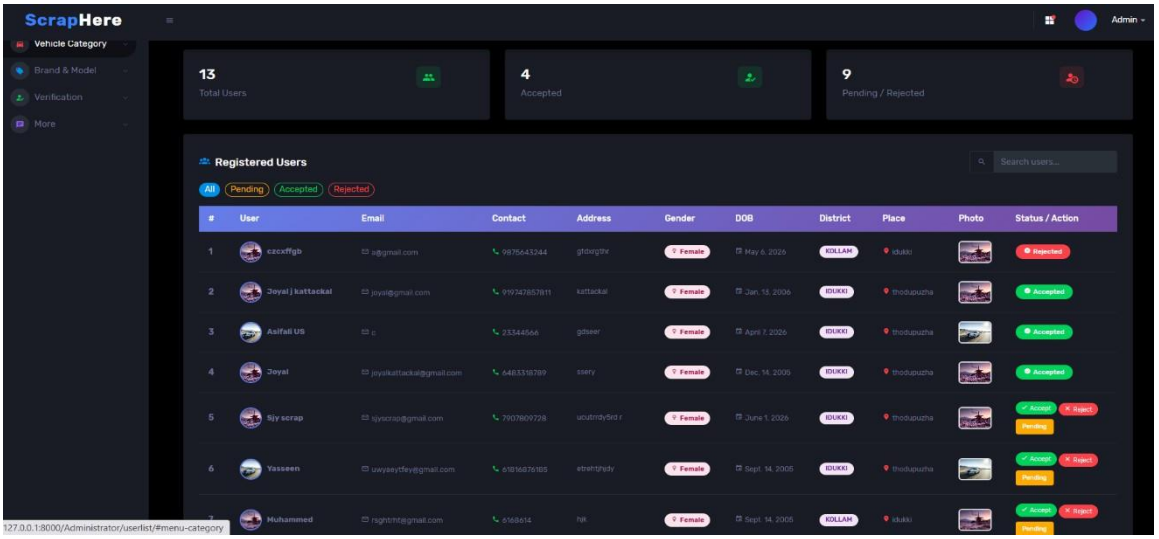
Login Page:



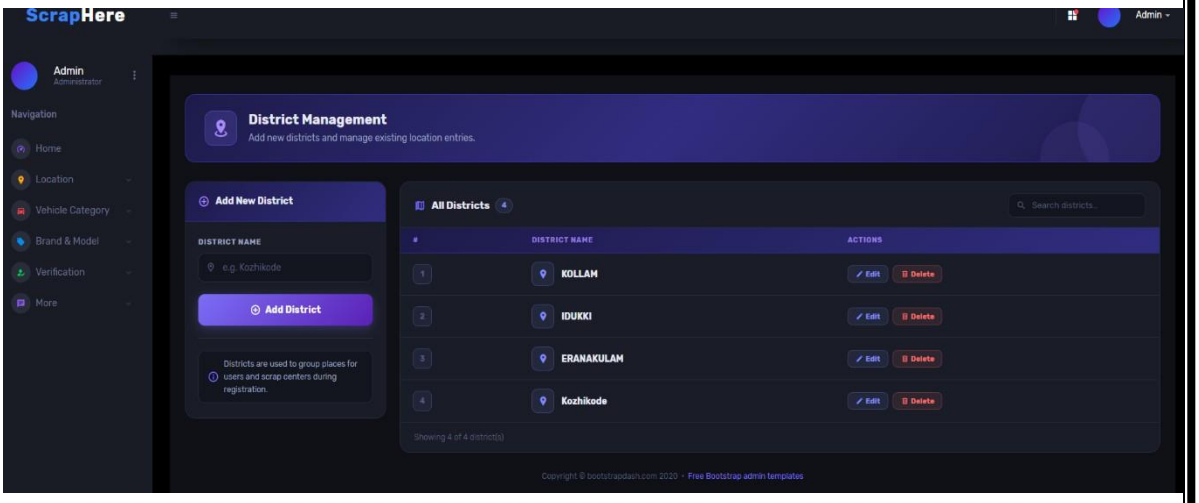
Home Page:



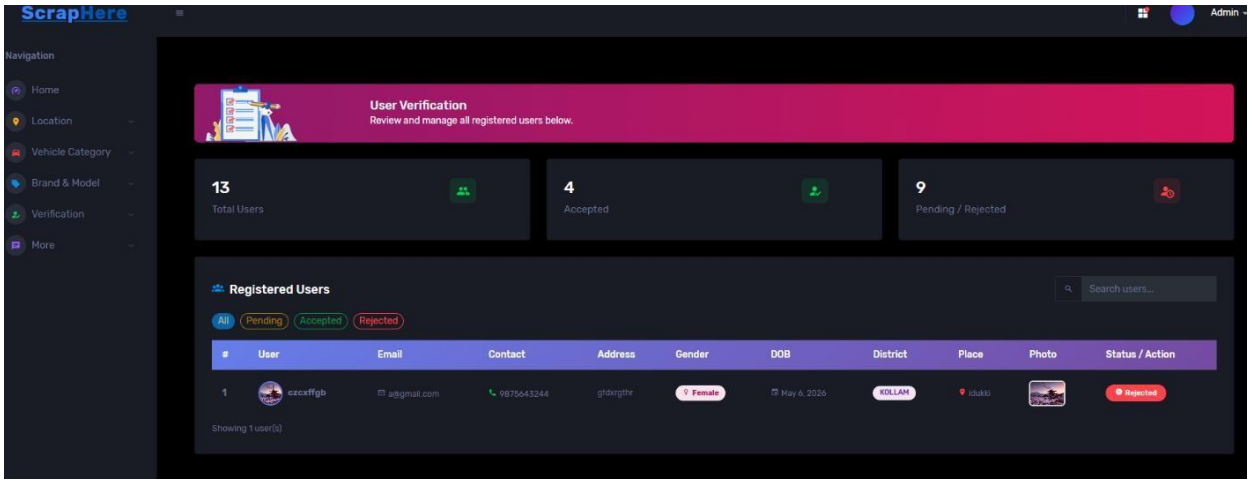
User List:



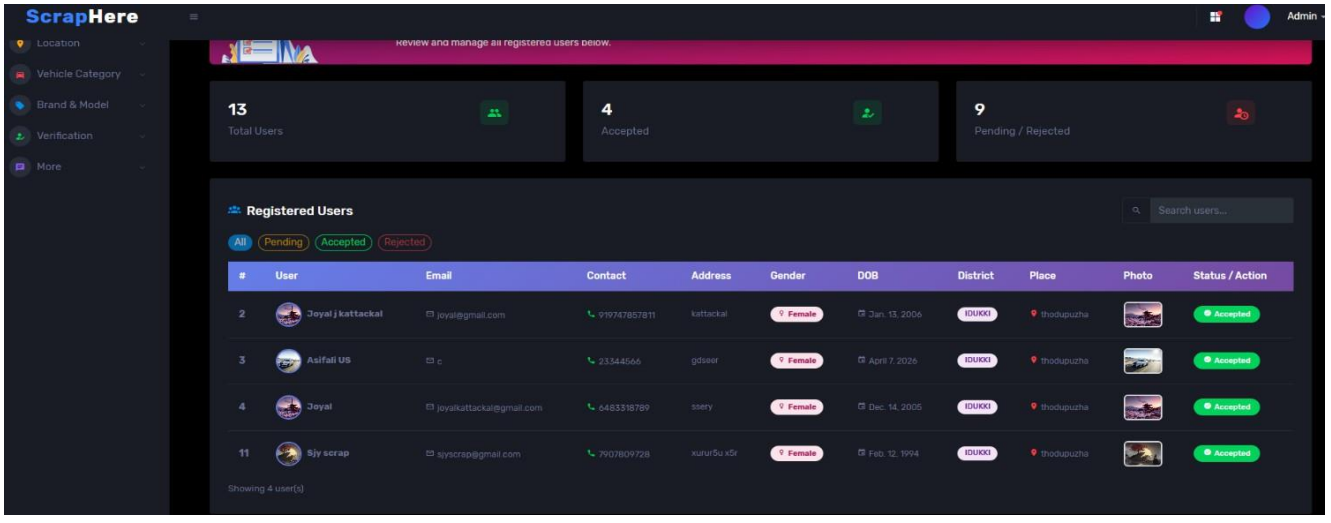
District Management:



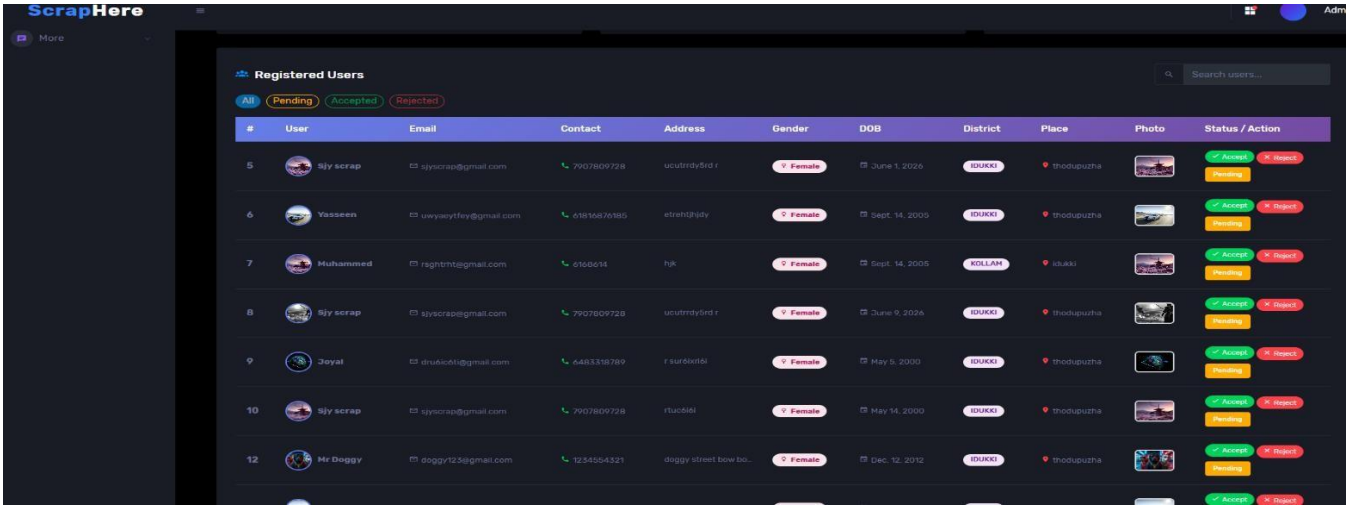
User Rejected List:



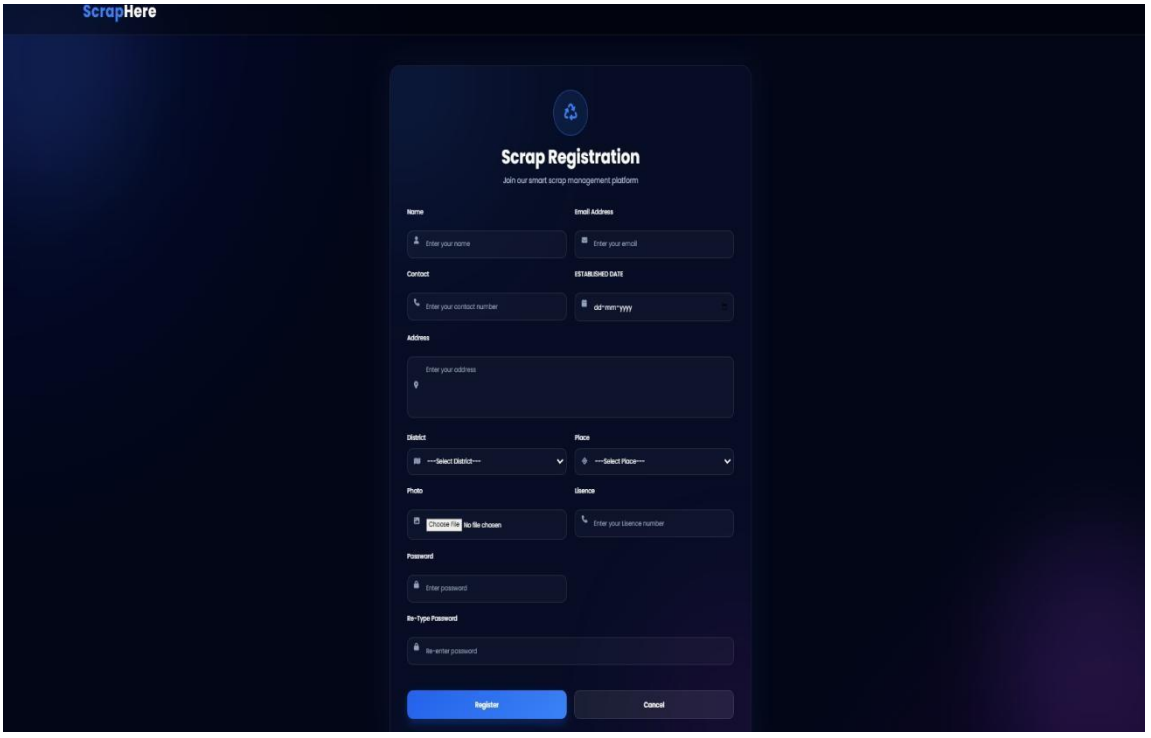
User Accepted List:



User Pending List

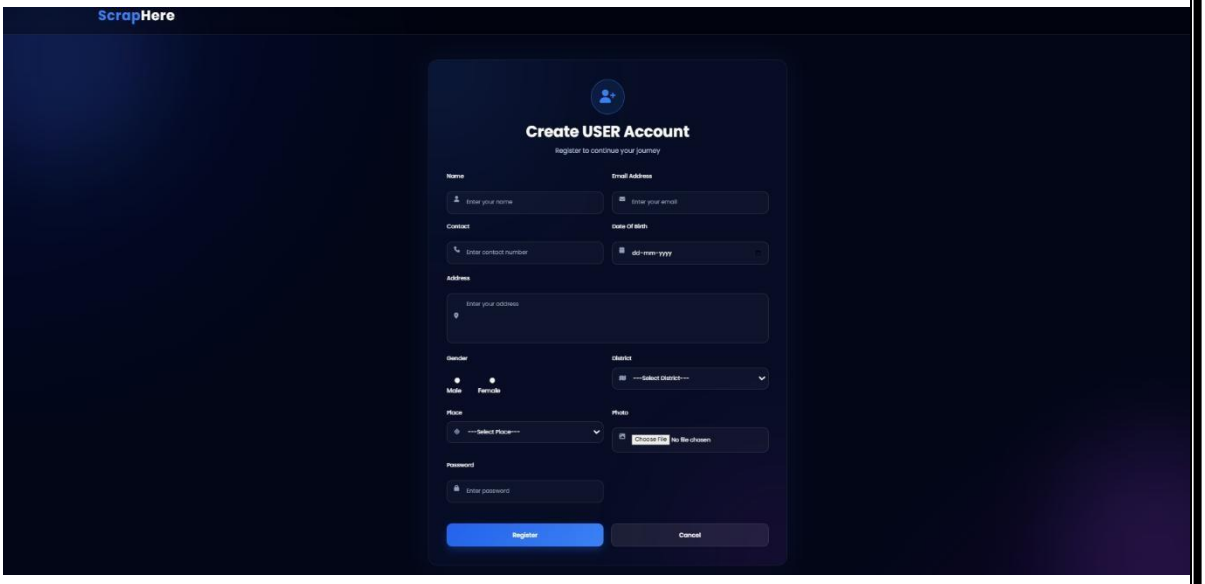


Scrap Registration:

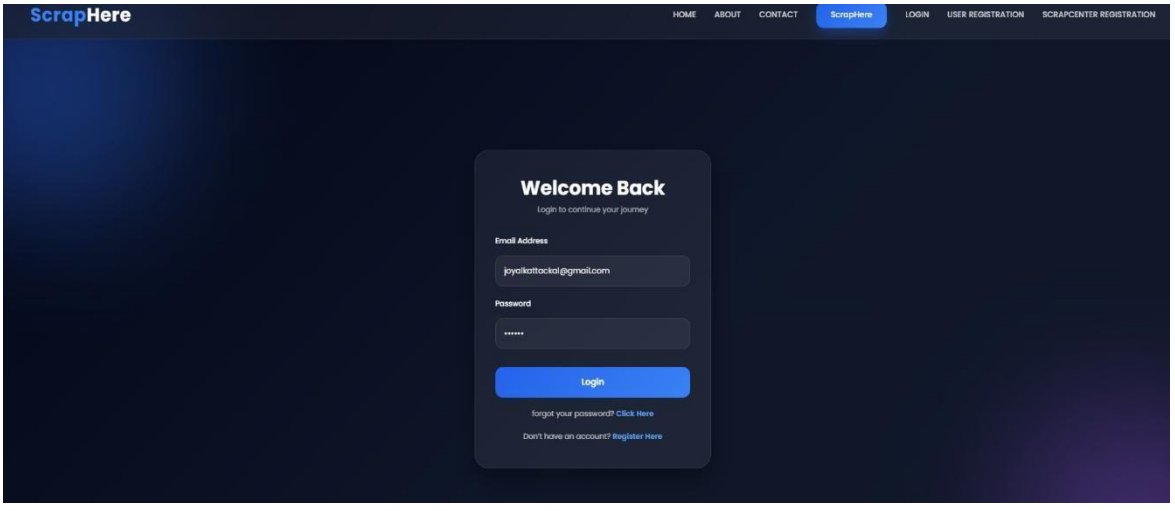


USER

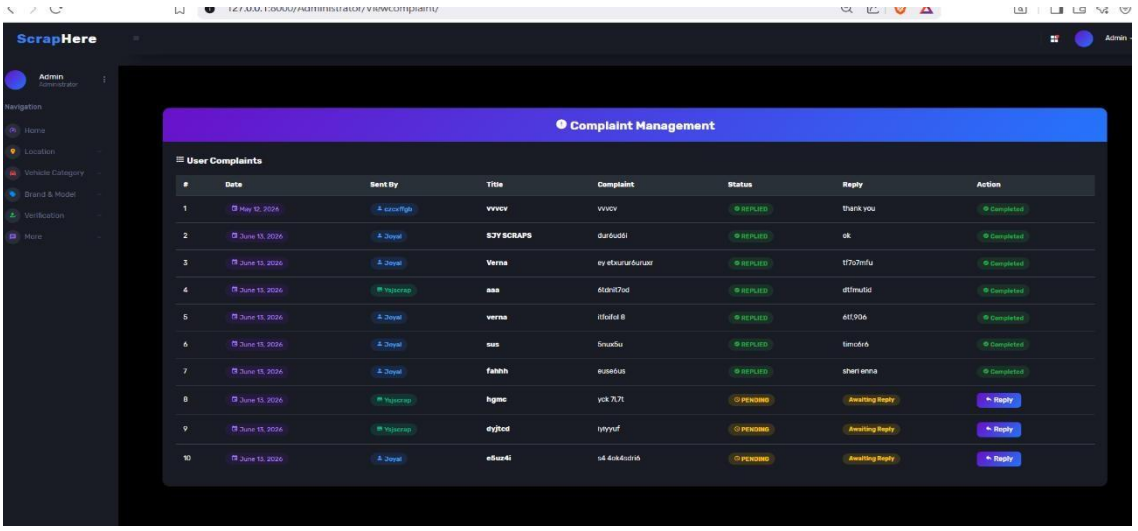
Registration:



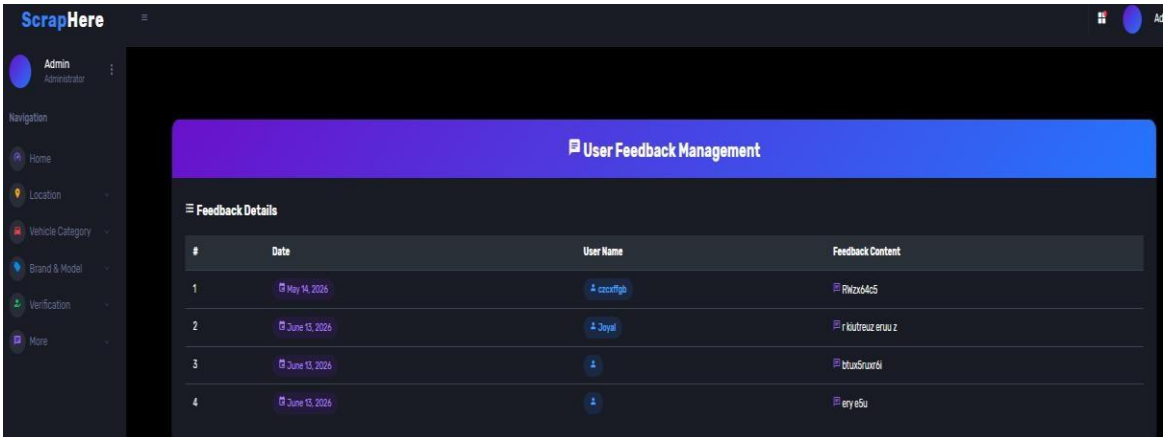
Login:



User Complaint:

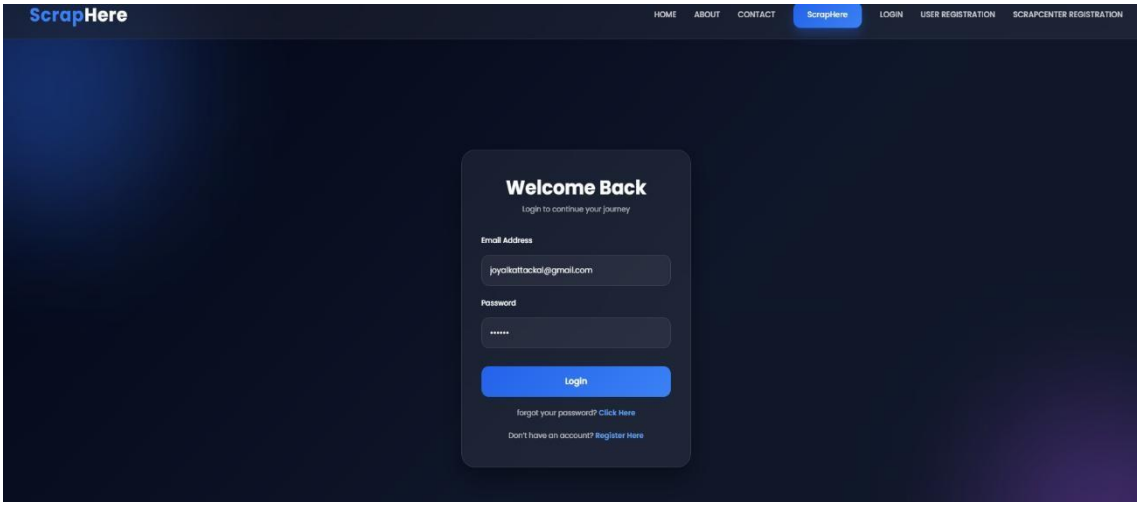


User Feedback:

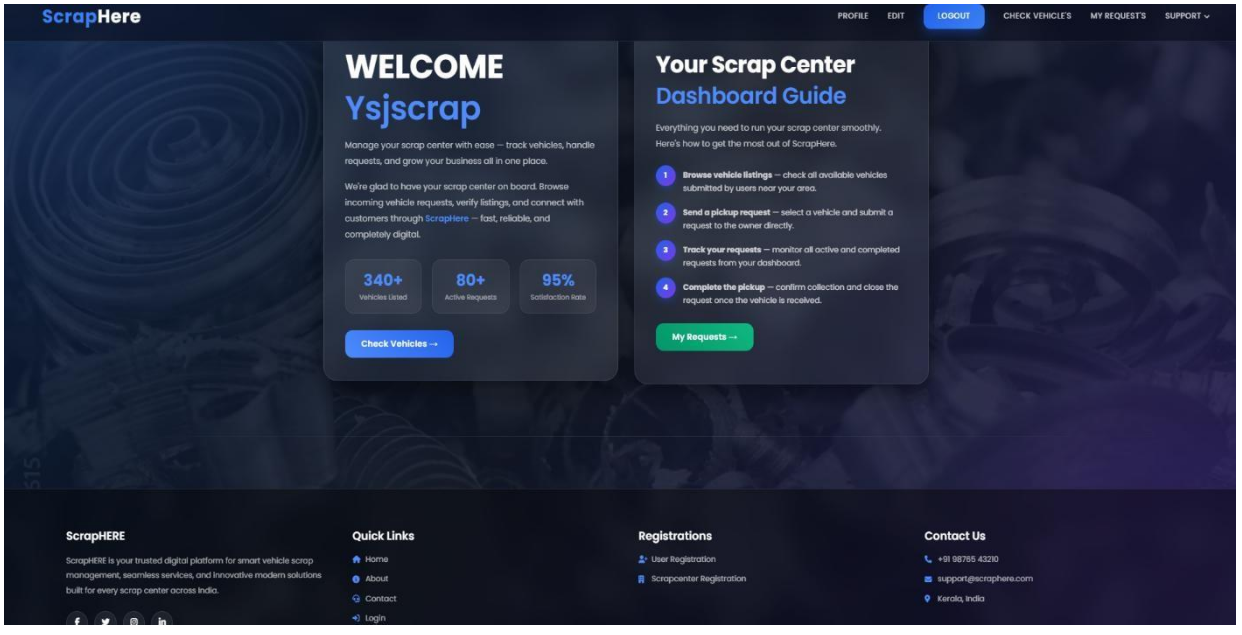


SCRAPCENTER

Login:

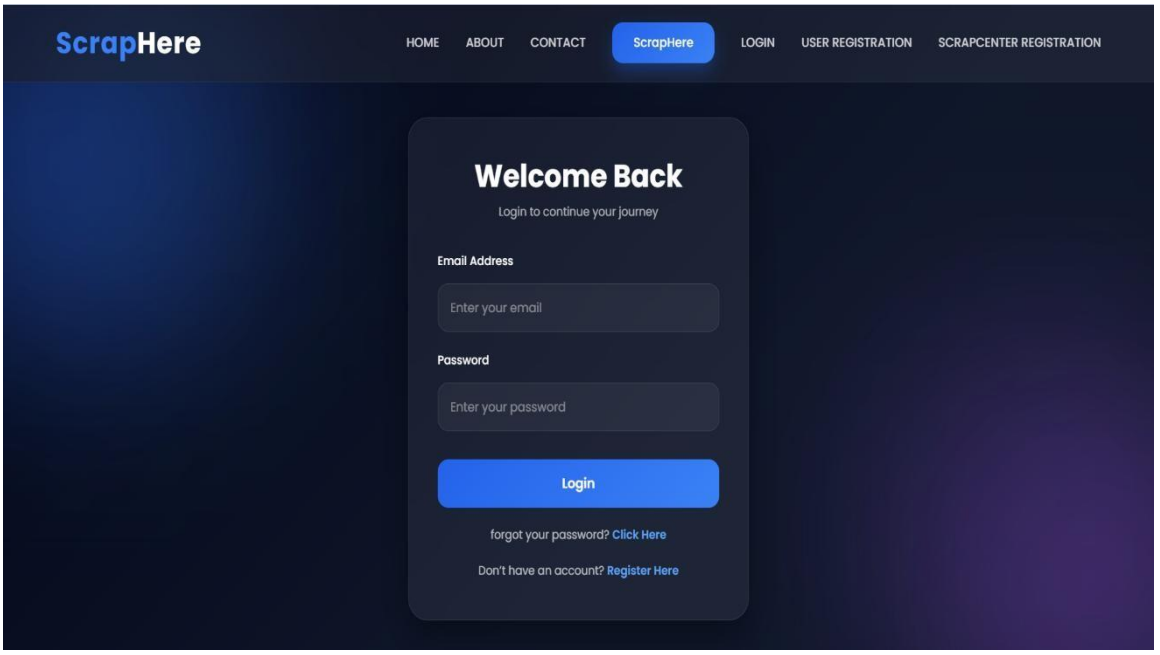


Home Page:

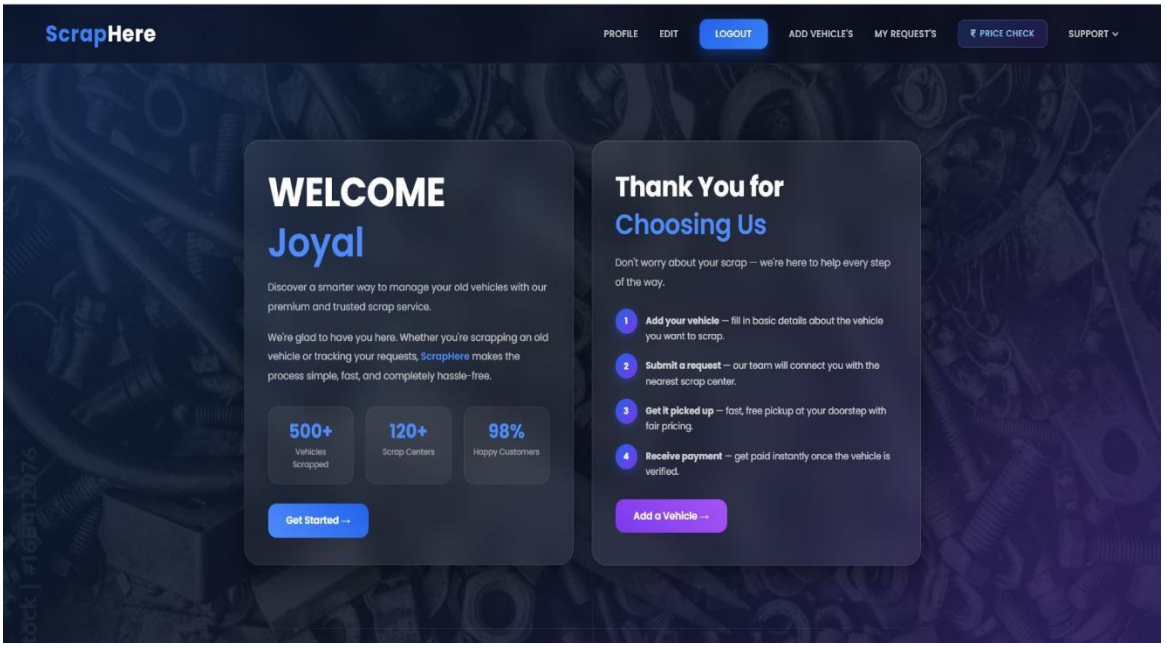


USER

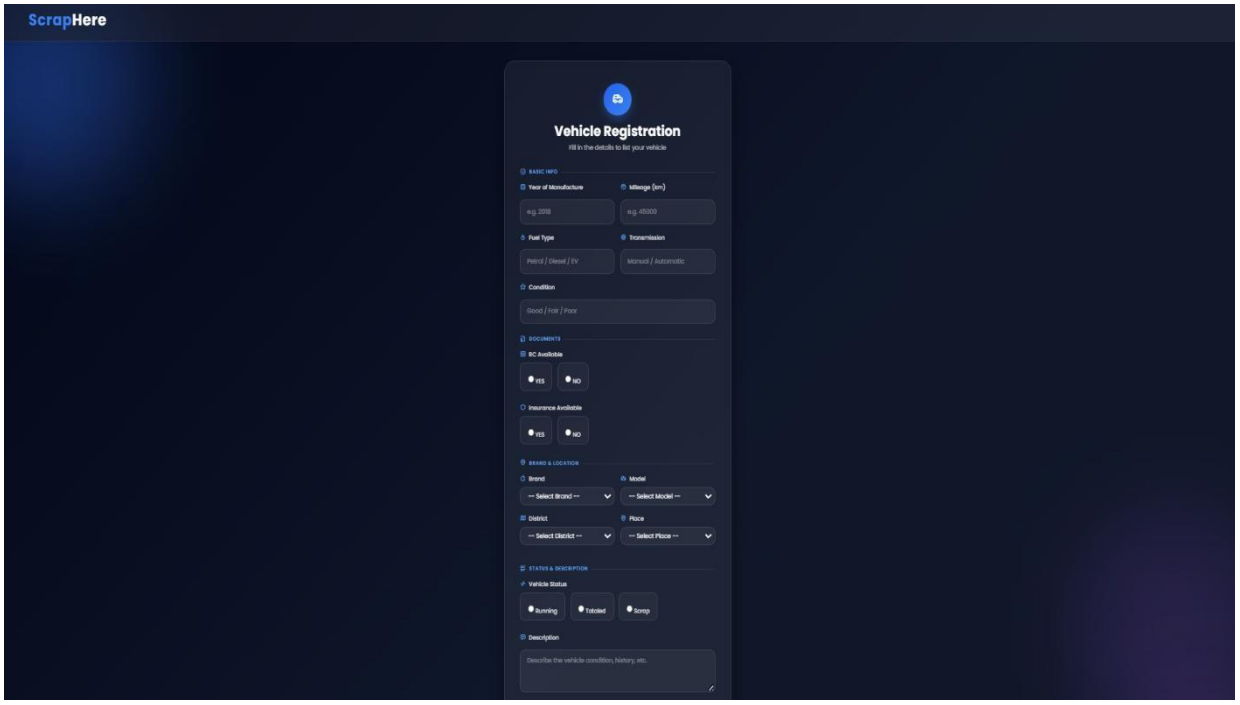
Login:



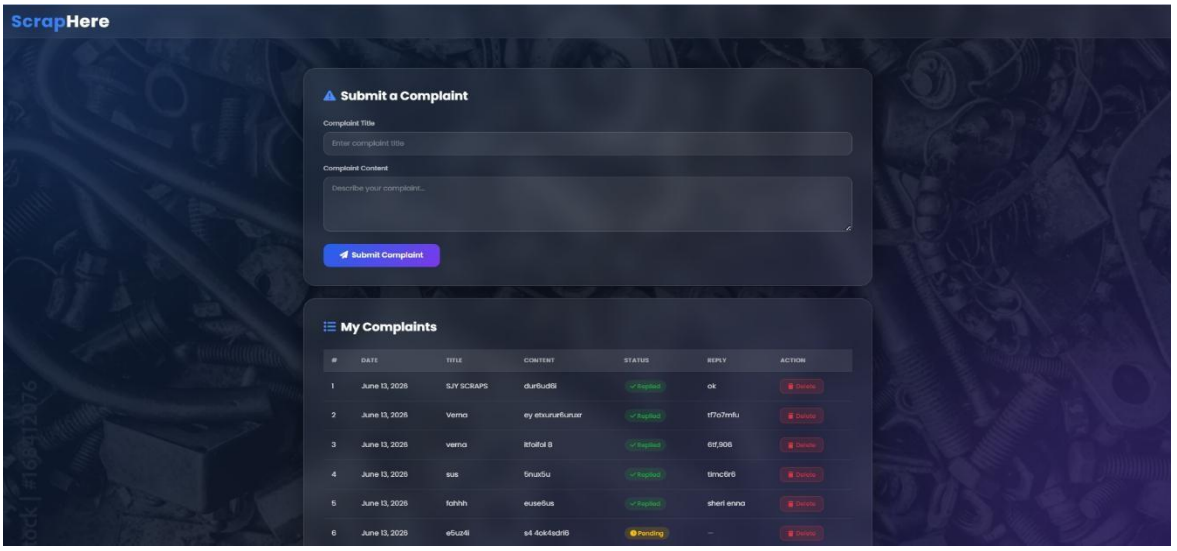
Home Page:



Vehicle Registration:



Complaint:



b. CODING

Login:

```
{% extends 'Guest/footer.html' %}
{% block content %}
<style>

/* ================= GOOGLE FONT ================= */

@import
url('https://fonts.googleapis.com/css2?family=Poppins:wght@300;400;500;600;700&display=s
wap');

*{
margin: 0;
padding: 0;
box-sizing: border-box;
}

/* ================= BODY ================= */

body{
font-family: 'Poppins', sans-serif;
background: linear-gradient(135deg,#020617,#0f172a,#111827);
overflow-x: hidden;
color: white;
}

/* ================= NAVBAR ================= */

.custom-navbar{
width: 100%;
padding: 18px 70px;
display: flex;
justify-content: space-between;
align-items: center;
position: sticky;
top: 0;
z-index: 999;
background: rgba(255,255,255,0.05);
backdrop-filter: blur(12px);
border-bottom: 1px solid rgba(255,255,255,0.08);
}

.logo{
font-size: 34px;
font-weight: 700;
color: #ffffff;
text-decoration: none;
}
.logo span{
color: #3b82f6;
```

```
.nav-links{
    display: flex;
    gap: 35px;
    align-items: center;
}

.nav-links a{
    text-decoration: none;
    color: rgba(255,255,255,0.8);
    font-size: 15px;
    font-weight: 500;
    transition: 0.3s;
    position: relative;
}

.nav-links a:hover{
    color: #60a5fa;
}

.nav-links a::after{
    content: '';
    position: absolute;
    width: 0%;
    height: 2px;
    background: #3b82f6;
    left: 0;
    bottom: -6px;
    transition: 0.3s;
}

.nav-links a:hover::after{
    width: 100%;
}

/* ACTIVE MENU */

.nav-active{
    background: linear-gradient(135deg,#2563eb,#3b82f6);
    padding: 12px 28px;
    border-radius: 14px;
    color: white !important;
    box-shadow: 0 10px 20px rgba(59,130,246,0.35);
}

/* ================= MAIN LOGIN SECTION ================= */

.login-section{
    min-height: 88vh;
    display: flex;
    justify-content: center;
    align-items: center;
    padding: 40px 15px;
    position: relative;
    overflow: hidden;
}
```

```
 * BACKGROUND GLOW */

.login-section::before{
    content: "";
    position: absolute;
    width: 420px;
    height: 420px;
    background: rgba(37,99,235,0.35);
    border-radius: 50%;
    top: -120px;
    left: -120px;
    filter: blur(120px);
}

.login-section::after{
    content: "";
    position: absolute;
    width: 350px;
    height: 350px;
    background: rgba(168,85,247,0.28);
    border-radius: 50%;
    bottom: -100px;
    right: -100px;
    filter: blur(120px);
}
/* ================= LOGIN CARD ================= */

.login-card{
    width: 100%;
    max-width: 450px;
    background: rgba(255,255,255,0.06);
    border: 1px solid rgba(255,255,255,0.10);
    backdrop-filter: blur(16px);
    border-radius: 26px;
    padding: 45px 38px;
    position: relative;
    z-index: 1;
    box-shadow: 0 20px 45px rgba(0,0,0,0.4);
    animation: fadeIn 0.7s ease;
}
@keyframes fadeIn{
    from{
        opacity: 0;
        transform: translateY(25px);
    }
    to{
        opacity: 1;
        transform: translateY(0);
    }
}
```

```
/* ================= LOGIN TITLE ================= */

.login-title{

text-align: center;
   margin-bottom: 35px;
}

.login-title h2{
   font-size: 36px;
   font-weight: 700;
   margin-bottom: 8px;
}

.login-title p{
   color: rgba(255,255,255,0.65);
   font-size: 14px;
}

/* ================= FORM ================= */

.form-group{
   margin-bottom: 22px;
}

.form-group label{
   display: block;
   margin-bottom: 8px;
   font-size: 14px;
   font-weight: 500;
   color: #fff;
}

.form-control{
   width: 100%;
   height: 55px;
   border-radius: 14px;
   border: 1px solid rgba(255,255,255,0.10);
   background: rgba(255,255,255,0.05);
   color: #fff;
   padding: 0 16px;
   font-size: 15px;
   transition: all 0.3s ease;
}

.form-control::placeholder{
   color: rgba(255,255,255,0.45);
}

.form-control:focus{
   outline: none;
```

```
border-color: #3b82f6;
    background: rgba(255,255,255,0.08);
    box-shadow: 0 0 18px rgba(59,130,246,0.35);
}

/* ================= BUTTON ================= */

.btn-login{
    width: 100%;
    height: 55px;
    border: none;
    border-radius: 14px;
    background: linear-gradient(135deg,#2563eb,#3b82f6);
    color: white;
    font-size: 16px;
    font-weight: 600;
    margin-top: 10px;
    transition: all 0.3s ease;
    cursor: pointer;
}

.btn-login:hover{
    transform: translateY(-3px);
    box-shadow: 0 12px 24px rgba(37,99,235,0.45);
}

/* ================= FOOTER TEXT ================= */

.login-footer{
    text-align: center;
    margin-top: 20px;
    color: rgba(255,255,255,0.7);
    font-size: 14px;
}

.login-footer a{
    color: #60a5fa;
    text-decoration: none;
    font-weight: 600;
}

.login-footer a:hover{
    color: #93c5fd;
}

/* ================= MAIN FOOTER ================= */

.main-footer{
    width: 100%;
    background: rgba(255,255,255,0.04);
    border-top: 1px solid rgba(255,255,255,0.08);
    backdrop-filter: blur(10px);
    padding-top: 60px;
}
```

```
.
footer-container{
    width: 90%;
    margin: auto;
    display: grid;
    grid-template-columns: repeat(auto-fit,minmax(220px,1fr));
    gap: 40px;
    padding-bottom: 40px;
}

.footer-box h3{
    margin-bottom: 18px;
    font-size: 22px;
    font-weight: 600;
}

.footer-box p{
    color: rgba(255,255,255,0.7);
    line-height: 1.8;
    font-size: 14px;
}

.footer-links{
    display: flex;
    flex-direction: column;
    gap: 12px;
}

.footer-links a{
    text-decoration: none;
    color: rgba(255,255,255,0.75);
    transition: 0.3s;
    font-size: 14px;
}

.footer-links a:hover{
    color: #60a5fa;
    transform: translateX(5px);
}

.footer-bottom{
    border-top: 1px solid rgba(255,255,255,0.08);
    text-align: center;
    padding: 18px;
    color: rgba(255,255,255,0.65);
    font-size: 14px;
}
```

```
/* ================= RESPONSIVE ================= */

@media(max-width:992px){

   .custom-navbar{
      padding: 18px 25px;
      flex-direction: column;
      gap: 20px;
   }

   .nav-links{
      flex-wrap: wrap;
      justify-content: center;
      gap: 20px;
   }
}

@media(max-width:576px){

   .login-card{
      padding: 35px 25px;
   }

   .login-title h2{
      font-size: 28px;
   }

   .logo{
      font-size: 28px;
   }
}

</style>

<!-- ================= NAVBAR ================= -->

<!-- ================= LOGIN SECTION ================= -->

<section class="login-section">

   <div class="login-card">

      <div class="login-title">
         <h2>Welcome Back</h2>
         <p>Login to continue your journey</p>
      </div>

      <form method="post">

         {% csrf_token %}
```

```
<!-- EMAIL -->

        <div class="form-group">

            <label>Email Address</label>

            <input type="email"
                   name="txt_Email"
                   class="form-control"
                   placeholder="Enter your email"
                   required>

        </div>

        <!-- PASSWORD -->

        <div class="form-group">

            <label>Password</label>

            <input type="password"
                   name="pswd_password"
                   class="form-control"
                   placeholder="Enter your password"
                   required>

        </div>

        <!-- BUTTON -->

        <button type="submit" class="btn-login">
            Login
        </button>

    </form>
      <div class="login-footer">

        forgot your password?

        <a href="{% url 'Guest:forgotpassword' %}">
            Click Here
        </a>

    <div class="login-footer">

        Don’t have an account?

        <a href="{% url 'Guest:userregistration' %}">
            Register Here
        </a>
```

```
</div>

   </div>

</section>


{% if msg %}
<script>
   alert("{{ msg }}");
   window.location = "{% url 'Guest:Login' %}";
</script>
{% endif %}
{% endblock content
```

Views:

```
def Login(request):
   if request.method == 'POST':
      email = request.POST.get("txt_Email")
      password = request.POST.get("pswd_password")

      UserCount = tbl_userregistration.objects.filter(
         userregistration_email=email,
         userregistration_password=password
      ).count()
      AdminCount = tbl_adminregistration.objects.filter(
         adminregistration_email=email,
         adminregistration_password=password
      ).count()
      ScrapcenterCount = tbl_ScrapcenterReg.objects.filter(
         scrapcenterreg_email=email,
         scrapcenterreg_password=password
      ).count()

      if UserCount > 0:
         userdata = tbl_userregistration.objects.get(
            userregistration_email=email,
            userregistration_password=password
         )
         if userdata.user_status == 0:
            return render(request, 'Guest/Login.html', {"msg": "Registration Pending"})
         elif userdata.user_status == 2:
            return render(request, 'Guest/Login.html', {"msg": "Registration Rejected"})
         else:
            request.session["uid"] = userdata.id
            return redirect("User:HomePage")

      elif ScrapcenterCount > 0:
         scrapdata = tbl_ScrapcenterReg.objects.get(
            scrapcenterreg_email=email,
            scrapcenterreg_password=password
         )
```

```
if scrapdata.scraper_status == 0:
            return render(request, 'Guest/Login.html', {"msg": "Registration Pending"})
        elif scrapdata.scraper_status == 2:
            return render(request, 'Guest/Login.html', {"msg": "Registration Rejected"})
        else:
            request.session["sid"] = scrapdata.id
            return redirect("Scrapcenter:HomePage")

    elif AdminCount > 0:
        admindata = tbl_adminregistration.objects.get(
            adminregistration_email=email,
            adminregistration_password=password
        )
        request.session["aid"] = admindata.id
        return redirect("Administrator:HomePage")

    else:
        return render(request, 'Guest/Login.html', {"msg": "User Does not Exist"})

else:
    return render(request, 'Guest/Login.html')
```

View Feedback:

```
{% extends 'Administrator/Header.html' %}
{% load static %}
{% block content %}

<style>
    .feedback-container {
        padding: 30px;
    }

    .feedback-card {
        background: #191c24;
        border-radius: 15px;
        box-shadow: 0 10px 25px rgba(0,0,0,0.3);
        overflow: hidden;
    }

    .card-header-custom {
        background: linear-gradient(135deg, #6a11cb, #2575fc);
        color: white;
        padding: 20px;
        text-align: center;
    }

    .card-header-custom h3 {
        margin: 0;
        font-weight: 600;
    }

    .table {
        color: white;
        margin-bottom: 0;
    }

    .table thead {
        background: #2A3038;
    }

    .table thead th {
        border: none;
        color: #fff;
        font-weight: 600;
    }

    .table tbody td {
        border-color: #3e444d;
        vertical-align: middle;
    }

    .table tbody tr:hover {
        background: rgba(255,255,255,0.05);
```

```
        }

        .section-title {
            color: white;
            font-weight: 600;
            margin-bottom: 20px;
        }

        .feedback-content {
            max-width: 400px;
            word-wrap: break-word;
        }

        .user-badge {
            background: rgba(37,117,252,0.15);
            color: #4da3ff;
            padding: 6px 12px;
            border-radius: 20px;
            font-size: 13px;
            font-weight: 500;
        }

        .date-badge {
            background: rgba(106,17,203,0.15);
            color: #b388ff;
            padding: 6px 12px;
            border-radius: 20px;
            font-size: 13px;
        }

        .btn-reply {
            background: linear-gradient(135deg, #6a11cb, #2575fc);
            color: white;
            border: none;
            border-radius: 8px;
            padding: 7px 16px;
            font-size: 12px;
            font-weight: 600;
            text-decoration: none;
            display: inline-flex;
            align-items: center;
            gap: 5px;
            transition: opacity 0.2s, transform 0.2s;
        }

        .btn-reply:hover {
            opacity: 0.85;
            transform: translateY(-1px);
            color: white;
            text-decoration: none;
        }
    </style>

    <div class="content-wrapper">
        <div class="feedback-container">
```

```
ssdiv class="card feedback-card">

        <div class="card-header-custom">
          <h3>
            <i class="mdi mdi-message-text"></i>
            User Feedback Management
          </h3>
        </div>

        <div class="card-body">

          <h4 class="section-title">
            <i class="mdi mdi-format-list-bulleted"></i>
            Feedback Details
          </h4>

          <div class="table-responsive">

            <table class="table table-hover">

              <thead>
                <tr>
                  <th>#</th>
                  <th>Date</th>
                  <th>User Name</th>
                  <th>Feedback Content</th>
                </tr>
              </thead>

              <tbody>

                {% for i in viewfeedback %}
                <tr>

                  <td>{{ forloop.counter }}</td>

                  <td>
                    <span class="date-badge">
                      <i class="mdi mdi-calendar"></i>
                      {{ i.feedback_date }}
                    </span>
                  </td>

                  <td>
                    <span class="user-badge">
                      <i class="mdi mdi-account"></i>
                      {{ i.user.userregistration_name }}
                    </span>
                  </td>

                  <td class="feedback-content">
                    <i class="mdi mdi-message-text-outline text-info"></i>
                    {{ i.feedback_content }}
                  </td>
                </tr>
```

```
        {% empty %}

                        <tr>
                          <td colspan="5" class="text-center text-muted">
                            No Feedback Available
                          </td>
                        </tr>

                        {% endfor %}

                      </tbody>

                    </table>

                  </div>

                </div>

              </div>

            </div>
          </div>

          {% endblock content %}
```

Views:

```
def viewfeedback(request):
    viewfeedback=tbl_feedback.objects.all()
    return render(request,'Administrator/ViewFeedback.html',{'viewfeedback':
viewfeedback})
```